Web Security Fundamentals: From Web Communication to Security Testing

Introduction

The World Wide Web is a complex system where multiple technologies work together to provide users with websites and web applications. Every interaction between a user and a web application involves communication between a client (browser) and a server.

Understanding how this communication works is one of the most important foundations in web security. Before testing vulnerabilities, a security professional must understand how requests are created, how responses are returned, how sessions are maintained, and how secure communication is established.

This assignment covers the fundamental concepts of web communication, including HTTP, HTTPS, TLS, certificates, browser behavior, and basic security testing using Burp Suite.

---

1. How the Web Works

Browser → Server Communication

When a user enters a URL into a browser, several steps happen before the webpage loads.

The communication flow is:

```
User Browser
      |
      ↓
DNS Resolution
      |
      ↓
TCP Connection
      |
      ↓
TLS Handshake (HTTPS)
      |
      ↓
HTTP Request
      |
      ↓
```

```
Web Server
      |
      ↓
HTTP Response
      |
      ↓
Browser Displays Content
```

DNS Resolution

DNS (Domain Name System) converts human-readable domain names into IP addresses.

Example:

```
www.example.com
        |
        ↓
93.184.216.34
```

The browser cannot communicate directly using domain names. It requires an IP address to locate the server.

TCP Connection

After obtaining the server IP address, the browser establishes a TCP connection.

TCP provides:

- Reliable communication
- Ordered data delivery
- Error checking

The TCP three-way handshake:

```
Client              Server

 SYN  ------------>

      <------------ SYN + ACK
```

```
ACK  ------------>
```

After this process, both sides can communicate.

---

2. HTTP Fundamentals

What is HTTP?

HTTP (HyperText Transfer Protocol) is a communication protocol used between browsers and web servers.

It defines how requests and responses are exchanged.

HTTP follows a:

```
Client → Request → Server

Server → Response → Client
```

model.

---

3. HTTP Request

An HTTP request is created by the browser when a user performs an action.

Example:

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: Firefox
Accept: text/html
```

A request contains:

Request Line

Example:

```
GET /login HTTP/1.1
```

It contains:

- HTTP method
- Requested resource
- HTTP version

Headers

Headers provide additional information about the request.

Examples:

Host Header

```
Host: example.com
```

Specifies which website the browser wants to access.

User-Agent

```
User-Agent: Mozilla/5.0
```

Identifies the browser and operating system.

Cookie Header

```
Cookie: session=abc123
```

Sends stored session information to the server.

---

4. HTTP Methods

HTTP methods define what action the client wants to perform.

GET

Used to retrieve information.

Example:

```
GET /profile
```

POST

Used to send data to the server.

Example:

```
POST /login
```

Login forms usually use POST requests.

PUT

Used to update existing data.

DELETE

Used to remove data.

---

5. HTTP Response

After receiving a request, the server sends a response.

Example:

```
HTTP/1.1 200 OK
```

```
Content-Type: text/html

<html>
<body>
Hello World
</body>
</html>
```

A response contains:

- Status code
- Headers
- Body

---

6. HTTP Status Codes

Status codes tell the client what happened.

200 OK

Request successful.

---

301 Redirect

Resource moved permanently.

---

302 Redirect

Temporary redirect.

---

304 Not Modified

The server tells the browser that the cached version can be used.

Example:

```
Browser:
Do you have a newer version?

Server:
No, use your cached copy.
```

This improves performance.

---

403 Forbidden

The server understands the request but refuses access.

---

404 Not Found

Requested resource does not exist.

---

500 Internal Server Error

Server-side problem.

---

7. Cookies and Sessions

Cookies

Cookies are small pieces of data stored in the browser.

They are commonly used for:

- Login sessions
- User preferences
- Tracking

Example:

```
Cookie:
```

```
session=78492abc
```

Cookie Security Flags

Secure Flag

A cookie with Secure flag is only sent over HTTPS.

Example:

```
Secure
```

HttpOnly Flag

Prevents JavaScript from accessing cookies.

Protection against some XSS attacks.

Example:

```
HttpOnly
```

SameSite Flag

Controls when cookies are sent with cross-site requests.

Values:

- Strict
- Lax
- None

Helps prevent CSRF attacks.

---

8. Sessions

HTTP itself is stateless.

It does not remember previous requests.

Sessions solve this problem.

Example:

```
User Login

↓


Server creates session

↓

Browser receives session cookie

↓

Future requests include session cookie
```

The server uses the session ID to identify the user.

---

9. HTTPS

HTTPS is HTTP running over encryption.

It provides:

Confidentiality

Data cannot be easily read by attackers.

Integrity

Data cannot be modified silently.

Authentication

The user can verify they are communicating with the correct server.

HTTPS uses TLS.

---

10. TLS Handshake

TLS creates a secure connection between browser and server.

Basic process:

```
Client                    Server

Client Hello  ----------->

              <----------- Server Hello

              <----------- Certificate

Key Exchange ------------>

Encrypted Communication
```

Client Hello

Browser sends:

- Supported TLS versions
- Supported encryption algorithms
- Random value

Server Hello

Server chooses:

- TLS version
- Encryption method

Certificate

Server sends its digital certificate.

The browser checks whether it is trusted.

---

11. Certificates and Certificate Authorities

Certificates prove the identity of websites.

Example:

```
Website Certificate

Issued To:
example.com

Issued By:
Trusted CA
```

Certificate Authorities (CA) are trusted organizations that verify website identities.

Examples:

- Let's Encrypt
- DigiCert

Browsers maintain a list of trusted CAs.
